

Code Explanation

YUYU Address Change Extract Transformations

The provided code implements business rule transformations for YUYU address change extract using PySpark. It defines a class `YUYUTransformations` that encapsulates various transformation methods.

Overview

This code is designed to apply a series of business rule transformations to a DataFrame containing YUYU address change extract data. The transformations are implemented as methods within the `YUYUTransformations` class, and are applied in a specific order to produce a transformed DataFrame ready for output.

Step 1: Initialize the YUYUTransformations Class

The `YUYUTransformations` class is initialized with a `Config` object, which is stored as an instance variable `self.config`. This configuration object is used later in the transformations.

```
class YUYUTransformations:
    def __init__(self):
        self.config = Config()
```

Step 2: Apply All Transformations

The `apply\_all\_transformations` method applies a series of business rule transformations to the input DataFrame. The transformations are applied in a specific order, and the resulting DataFrame is returned.

```
def apply_all_transformations(self, df: DataFrame) -> DataFrame:
    df = self._apply_reception_date_formatting(df) # BR-03
    df = self._apply_address_fallback_logic(df)    # BR-04
    df = self._apply_postal_code_normalization(df) # BR-05
    df = self._apply_phone_formatting(df)          # BR-06
    df = self._apply_owner_name_derivation(df)      # BR-07, BR-08
    df = self._apply_process_stamps(df)            # BR-01, BR-02
    df = self._add_sequence_number(df)
    return df
```

Step 3: Apply Reception Date Formatting

The `\_apply\_reception\_date\_formatting` method applies the reception date formatting transformation (BR-03). It converts the `T\_TX\_REQUEST\_REQ\_ACC\_DATETIME` column to a timestamp using the `to\_timestamp` function.

```
def _apply_reception_date_formatting(self, df: DataFrame) -> DataFrame:
    return df.withColumn(
        "RECEPTION_DATE",
        to_timestamp(
            date_format(col("T_TX_REQUEST_REQ_ACC_DATETIME"), "yyyy-MM-dd HH:mm"),
            "yyyy-MM-dd HH:mm"
```

```
)  
)
```

Step 4: Apply Address Fallback Logic

The `\_apply\_address\_fallback\_logic` method applies the address fallback logic transformation (BR-04). It checks if the ZIP code is missing, and if so, uses the `PPAY\_\*` columns as a fallback for the new address Kana fields.

```
def _apply_address_fallback_logic(self, df: DataFrame) -> DataFrame:  
    zip_missing = (  
        col("T_TX_DTL_ADD_CH_N_TRANS_ZIP").isNull() |  
        (trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP")) == "") |  
        (length(trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP"))) == 0)  
    )  
  
    df = df.withColumn(  
        "NEW_ADDRESS_KANA_1",  
        when(zip_missing, col("PPAY_ADR1_FW"))  
        .otherwise(col("T_TX_DTL_ADD_CH_N_TRANS_ADD1"))  
    )  
    # ...  
    return df
```

Step 5: Apply Postal Code Normalization

The `\_apply\_postal\_code\_normalization` method applies the postal code normalization transformation (BR-05). It removes hyphens from the postal code and falls back to `PPAY\_ZIP` if the main ZIP code is missing.

```
def _apply_postal_code_normalization(self, df: DataFrame) -> DataFrame:  
    zip_missing = (  
        col("T_TX_DTL_ADD_CH_N_TRANS_ZIP").isNull() |  
        (trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP")) == "") |  
        (length(trim(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP"))) == 0)  
    )  
  
    df = df.withColumn(  
        "v_zip",  
        when(zip_missing, col("PPAY_ZIP"))  
        .otherwise(col("T_TX_DTL_ADD_CH_N_TRANS_ZIP"))  
    )  
  
    df = df.withColumn(  
        "NEW_ADDRESS_POSTAL_CODE",  
        regexp_replace(coalesce(col("v_zip"), lit("")), "-", "")  
    )  
  
    return df.drop("v_zip")
```

Step 6: Apply Phone Formatting

The `\_apply\_phone\_formatting` method applies the phone formatting transformation (BR-06). It formats

the telephone number according to specific rules for mobile and landline numbers.

```
def _apply_phone_formatting(self, df: DataFrame) -> DataFrame:
    phone_col = trim(col("T_TX_DTL_ADD_CH_N_TRANS_PHNO"))

    # Check if NULL or blank
    phone_is_blank = phone_col.isNull() | (phone_col == "")

    # Extract prefix (first 3 chars)
    prefix = substring(phone_col, 1, 3)
    is_mobile = prefix.isin(["050", "060", "070", "080", "090"])

    # Mobile format: AAA-BBBB-CCCCC
    mobile_format = concat(
        substring(phone_col, 1, 3),
        lit("-"),
        substring(phone_col, 4, 4),
        lit("-"),
        substring(phone_col, 8, 6)
    )

    # Landline format: AA-BBBB-CCCCC
    landline_format = concat(
        substring(phone_col, 1, 2),
        lit("-"),
        substring(phone_col, 3, 4),
        lit("-"),
        substring(phone_col, 7, 6)
    )

    df = df.withColumn(
        "NEW_ADDRESS_TELEPHONE_NUMBER",
        when(phone_is_blank, lit(""))
        .when(is_mobile, mobile_format)
        .otherwise(landline_format)
    )

    return df
```

Step 7: Apply Owner Name Derivation

The `\_apply\_owner\_name\_derivation` method applies the policy owner name derivation transformations (BR-07 and BR-08). It derives the policy owner name in Kana and Kanji.

```
def _apply_owner_name_derivation(self, df: DataFrame) -> DataFrame:
    first_name_blank = (
        col("POWN_FNM").isNull() |
        (trim(col("POWN_FNM")) == "")
    )

    df = df.withColumn(
        "POLICY_OWNER_NAME_KANA",
        when(
            first_name_blank,
            trim(col("POWN_LNM"))
        ).otherwise(
            concat(

```

```

        trim(col("POWN_LNM")),
        lit(" "),
        trim(col("POWN_FNM"))
    )
)
df = df.withColumn(
    "POLICY_OWNER_NAME_KANJI",
    col("POWN_KNM")
)
return df

```

Step 8: Apply Process Stamps

The `\_apply\_process\_stamps` method applies the process stamps transformations (BR-01 and BR-02). It adds the processing date and user ID to the DataFrame.

```

def _apply_process_stamps(self, df: DataFrame) -> DataFrame:
    df = df.withColumn("STG_PROCESS_DATE", current_timestamp()) # B
R-01
    df = df.withColumn("STG_PROCESS_USERID", lit(self.config.M_PROCE
SS_USERID)) # BR-02
    return df

```

Step 9: Add Sequence Number

The `\_add\_sequence\_number` method adds a sequence number to the DataFrame, starting from 0 with an increment of 1.

```

def _add_sequence_number(self, df: DataFrame) -> DataFrame:
    window_spec = Window.orderBy(lit(1))

    df = df.withColumn(
        "SEQUENCE_NUMBER",
        row_number().over(window_spec) - 1
    )
    return df

```

Step 10: Select Flat File Columns

The `\_select\_flat\_file\_columns` method selects and orders the columns for the DPAddressChangeYUYU output.

```

def select_flat_file_columns(self, df: DataFrame) -> DataFrame:
    return df.select(
        col("SEQUENCE_NUMBER").cast("int"),
        col("T_TX_REQUEST_ORIGIN_REQUEST_ID").alias("ORIGIN_REQUEST_
ID"),
        col("T_TX_BASIC_POLICY_ID").alias("POLICY_ID"),

```

```
col("RECEPTION_DATE"),  
col("NEW_ADDRESS_POSTAL_CODE"),  
col("NEW_ADDRESS_KANA_1"),  
col("NEW_ADDRESS_KANJI_1"),  
col("NEW_ADDRESS_KANA_2"),  
col("NEW_ADDRESS_KANJI_2"),  
col("NEW_ADDRESS_KANA_3"),  
col("NEW_ADDRESS_KANJI_3"),  
col("NEW_ADDRESS_TELEPHONE_NUMBER"),  
col("POLICY_OWNER_NAME_KANA"),  
col("POLICY_OWNER_NAME_KANJI")  
)
```